

DESARROLLO WEB EN ENTORNO SERVIDOR
TÉCNICO EN DESARROLLO DE APLICACIONES WEB

Generación dinámica de páginas Web

ÍNDICE

/ 1. Introducción y contextualización práctica	3
/ 2. La generación dinámica	4
2.1. Repaso de conceptos generales de PHP	4
/ 3. Redirección de páginas en PHP I. Diseño del origen	5
3.1. El objeto Location. Parámetros y métodos	6
/ 4. Redirección de páginas en PHP II. Diseño del destino	7
4.1. Acceso a parámetros de envío y diseño pantalla origen	7
4.2. Diseño pantalla destino	8
/ 5. Obtención remota de la información	8
/ 6. Controles con verificación de información remota en el cliente	9
6.1. Filtros de validación y saneamiento	10
6.2. Filter_input y filter_has_var	11
6.3. Diseño de un caso de verificación con formulario	12
/ 7. Caso práctico 1: “Uso del elemento Location”	13
/ 8. Caso práctico 2: “Crea un formulario verificador”	13
/ 9. Modificación de la estructura de una página	14
/ 10. Resumen y resolución del caso práctico de la unidad	15
/ 11. Bibliografía	16

OBJETIVOS

Identificar las librerías y las tecnologías relacionadas con la generación por parte del servidor de páginas Web con guiones embebidos.

Utilizar estas tecnologías para generar páginas Web que incluyan interacción con el usuario en forma de advertencias y peticiones de confirmación.

Utilizar estas tecnologías para generar páginas Web que incluyan verificación de formularios.

Utilizar estas tecnologías para generar páginas Web que incluyan modificación dinámica de su contenido y su estructura.

Aplicar estas tecnologías en la programación de aplicaciones Web.

/ 1. Introducción y contextualización práctica

Las tecnologías recogidas en el capítulo anterior permiten la construcción de páginas web interactivas. Para definir el concepto de página web interactiva no solo debemos pensar en un sitio web “dinámico” en cuanto a su diseño visual, sino que se refiere a un **sitio en el cual exista una interacción activa entre cliente y servicio, dónde el usuario realizará diferentes tipos de peticiones y la página web le devolverá la respuesta oportuna.**

La redirección de páginas a través del lenguaje de programación PHP, resulta una herramienta clave en el desarrollo de páginas web interactivas. En este capítulo nos centramos en el uso del objeto *Location*, analizando sus funciones y propiedades principales a través de varios ejemplos de uso.

Finalmente, se explorarán los filtros de verificación y saneamiento de datos que incorpora PHP en su manual de desarrollo.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema, encontrarás su resolución en el apartado Resumen y Resolución del caso práctico.



Fig. 1. Imagen ilustrativa



Audio intro. “Verificación de datos en PHP”

<https://bit.ly/38bVN7k>





/ 2. La generación dinámica

Como hemos adelantado, una página web interactiva es aquel **sitio en el cual existe una interacción activa entre cliente y servicio, donde el usuario realizará diferentes tipos de peticiones y la página web le devolverá la respuesta oportuna.**

Este comportamiento interactivo incluirá peticiones a servidor habitualmente, y a base de datos, que devolverán diferentes respuestas en función de la petición y del usuario. Uno de los usos más habituales está dirigido a la **autenticación del usuario**. En estos casos si los datos son correctos y están contenidos en alguna tabla de la base de datos, el usuario podrá acceder a un sitio web y además mantener el estado de las sesiones anteriores. El correcto uso de las bases de datos es imprescindible para el desarrollo de sitios web interactivos.

Ahora bien, otra de las implementaciones habituales en la construcción de páginas web interactivas, es permitir al usuario navegar por todo el sitio web en función de la interacción que éste realice con la página.

En este capítulo, al igual que en el resto del módulo, nos vamos a centrar en el funcionamiento de este tipo de páginas utilizando el **lenguaje de programación PHP**.

Como hemos visto anteriormente, los archivos desarrollados utilizando lenguaje PHP tienen que almacenarse utilizando la extensión **.php**. Este tipo de archivos contienen las instrucciones codificadas con PHP y que aparecen embebidas con los lenguajes propios del desarrollo web, como son HTML, JavaScript o CSS.

Recordamos que para poder embeber un fragmento de código o fichero PHP en un documento HTML se utilizan las **etiquetas de cierre y apertura <?php y ?>**. Todo código que se escriba fuera de estas etiquetas será ignorado por PHP, pero sí tratará de ser interpretado por HTML.

En PHP, a diferencia de HTML, es necesario separar cada instrucción incorporando al final de la línea el **símbolo; (punto y coma)**. Puede omitirse en la última línea de un bloque de código PHP.



Fig. 2. Logo PHP

2.1. Repaso de conceptos generales de PHP

De nuevo, utilizaremos el entorno de **desarrollo XAMPP** analizado ampliamente en los primeros capítulos de este módulo. **Todos los ficheros PHP serán colocados en la carpeta htdocs**, desde la cual **podrán ser ejecutados en el navegador** siempre que el servidor esté encendido.



Fig. 3. Ficheros y estructura de carpetas



1. Desde la carpeta **htdocs** se crea una nueva carpeta para cada proyecto nuevo. Aunque esto no es completamente necesario, puesto que podríamos crear un nuevo fichero sin necesidad de crear más carpetas, pero de esta manera el resultado queda mucho más organizado.
2. A continuación, **se implementa el código de desarrollo del proyecto**. Dentro de las etiquetas `<body>` colocamos la propias de PHP como se muestra en el siguiente esqueleto.

```
<html>
<head>...</head>
<body>
  <?php
    //CÓDIGO PHP
  ?>
</body>
</html>
```

Código 1. Código base de inclusión php en html

3. Finalmente, **se guarda el fichero añadiendo la extensión .php al nombre del archivo**.
4. Desde la barra del navegador accedemos a la URL en la que se aloja la página PHP. Si se ha situado dentro de alguna carpeta recuerda que tienes que indicar la ruta exacta de la siguiente manera:
<http://localhost/carpeta1/carpeta2/.../nombre.php>

/ 3. Redirección de páginas en PHP I. Diseño del origen

La redirección de páginas a través del lenguaje de programación PHP, resulta una **herramienta clave en el desarrollo de páginas web interactivas**. Para implementar este funcionamiento será necesario desarrollar tantos ficheros PHP como páginas o secciones se quieran mostrar a un usuario ante las tomas de decisión, y la interacción que éste realiza sobre el sitio web.

Ahora bien ¿cómo **se implementa** esta redirección?: **Utilizando lenguaje de programación JavaScript**. A través del uso de la librería JavaScript, será posible definir ciertas funciones que permitan la redirección de una página a otra.

Para esta redirección es imprescindible el uso de un nuevo tipo de objeto, conocido como **Location (windos. location)** que será el encargado de implementar la redirección a otra página web. Algunas de las propiedades de este objeto son:

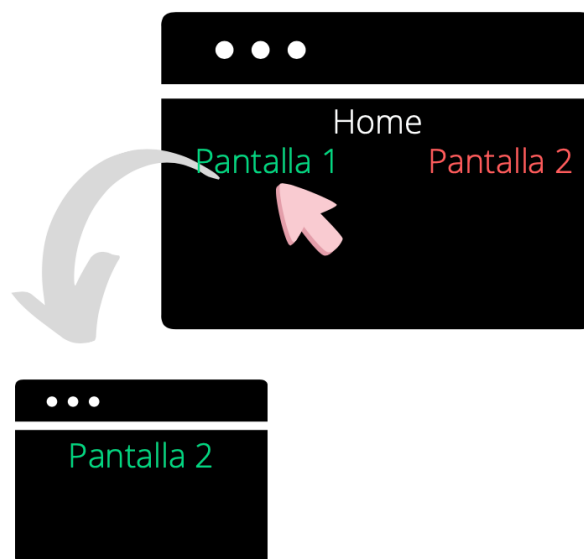


Fig. 4. Diagrama comportamiento interactivo

Propiedad	Definición
href	Indica en un string con la url complete.
protocol	Indica el protocolo de comunicación utilizado.
pathname	Indica el nombre del fichero concreto que se está ejecutando.
search	Indica el nombre y el valor de los parámetros que acompañan a la url.

Tabla 1. Propiedades objeto Location



Por lo tanto, para realizar la redirección a una nueva página web se utilizará una sintaxis como la que se muestra en el código de ejemplo siguiente:

```
location.href='nombreDestino.php';
```

Código 2. Código location en php en html

3.1. El objeto Location. Parámetros y métodos

Paso de parámetros

Además de indicar el fichero destino (nombreDestino.php) a través de la propiedad **href** del objeto **Location**, será posible el **envío de variables de forma opcional**.

Para ello se coloca al final del nombre del fichero, justo detrás de la extensión php, los valores que van a ser enviados, separados por un símbolo de interrogación '?'.

```
location.href='nombreDestino.php?nombreVariable=valorVariable';
```

Código 3. Código location href con parámetros

Como se puede ver en el siguiente fragmento de código, **se ha de indicar el nombre de la variable y el valor de la misma**. De esta forma, cuando en el destino se quiera acceder a un determinado valor, será posible hacerlo indicando el nombre de la variable en la que está contenido.

En los siguientes apartados se verá cómo acceder a los valores de las variables desde el fichero de destino.

Métodos

Al tratarse de un objeto, *Location* incluye una librería con algunas funciones permiten realizar diferentes operaciones sobre la instalación del objeto.

Método	Definición
assign (url)	Esta función asigna una url a la ventana sobre la que es implementado el objeto Location. Para volver a la ventana anterior basta con pulsar la flecha hacia atrás de cualquier navegador.
reload(boolean)	Si el valor es true permite la recarga de la ventana desde el servidor, si es false o no se indica ningún valor, se utilizará la última cargada en el memoria cache.
replace (url)	Esta función asigna una url a la ventana sobre la que es implementado el objeto Location. No es posible regresar a la ventana anterior.
toString()	Esta función devuelve como cadena de texto el valor de la url de la ventana.

Tabla 2. Funciones Location



Video 1. "Análisis objeto Location y ejemplo"
<https://bit.ly/3kYwYiu>





/ 4. Redirección de páginas en PHP II. Diseño del destino

4.1. Acceso a parámetros de envío y diseño pantalla origen

En los ficheros PHP de destino se podrá acceder al contenido de las variables enviadas en la redirección utilizando el array `$_GET[]` e indicando **entre los corchetes el nombre que identifica la variable a la que se va a acceder**.

```
$_GET [ nombreVariable ]
```

Código 4. Acceso a parámetros GET de HTTP

Por ejemplo, si se realiza un envío del fichero “primerapantalla.php” y se desea que, además, se adjunte una variable de nombre “identificador” y cuyo valor es “404”, la sintaxis de envío quedaría de la siguiente forma:

```
location.href='primerapantalla.  
php?identificador=404'
```

Código 5. Ejemplo location href

Para ilustrar todo el proceso analizado en el apartado 3, en base al objeto *Location*, así como la recepción de parámetros anteriormente mencionada, **se diseñan** a continuación dos ficheros correspondientes a **dos páginas web o pantallas**, la primera será el **punto de origen** y la segunda, **el destino que recibe los parámetros enviados por la primera**.

Para el de **punto de origen**, en primer lugar, en el siguiente fichero de origen (Pantalla 1) si el usuario pulsa el botón “Continuar” será redireccionado a una nueva página (Pantalla 2), mientras que si no pulsa dicho botón se mantendrá en la página de inicio (Pantalla 1).

```
<html>  
<head>  
<title> Pantalla 1 - ORIGEN </title>  
<script type="text/javascript">  
function siguientePantalla(){  
if(confirm('¿Estás seguro?')){  
location.href='pantalla2.php?valor=0';  
}  
}  
</script>  
</head>  
<body>  
<p onclick="siguientePantalla()"> Continuar </p>  
</body>  
</html>
```

Código 6. Diseño HTML y Location

4.2. Diseño pantalla destino

Hasta aquí se habría implementado la primera de las pantallas, la cual permite acceder a la segunda cuando se pulsa el botón *Continuar*, siempre y cuando se haya creado un fichero donde se construye la pantalla destino.

Cómo se puede observar en el código 6, aparece la función **confirm()** de JavaScript, que muestra por pantalla un mensaje de alerta que permite confirmar si la acción se va a llevar a cabo o no. Es decir, en este caso, cuándo se pulsa el botón *Continuar*, aparecerá una nueva ventana de diálogo con el texto “¿Estás seguro?” y dos botones, *Aceptar* y *Cancelar*. Si se pulsa **Aceptar**, se ejecuta el código colocado dentro de la estructura de control **if**.

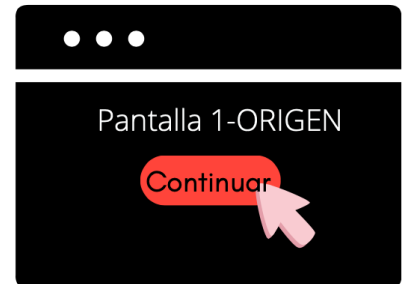


Fig. 5. Diseño pantalla destino (I)

El fichero de destino que define la Pantalla 2 se expone a continuación, en el que se hace uso de la función `$_GET[]`.

```
<html>
<head>
<title> Pantalla 2 - DESTINO </title>
</head>
<body>
<?php
$valorOrigen = $_GET["valor"];
echo "Pantalla 2 - Valor: ".$valorOrigen);
?>
</body>
</html>
```

Código 7. Diseño datos pantalla destino con Location y GET

Si todo ha funcionado correctamente, la pantalla 1 de origen, dará lugar a la pantalla 2 de destino.

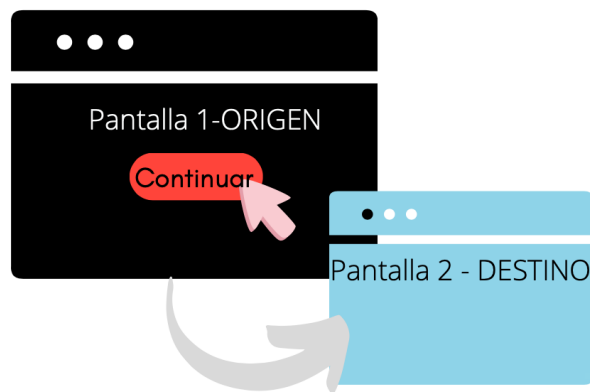


Fig. 6. Diseño pantalla destino (II)

/ 5. Obtención remota de la información

La recuperación de datos almacenados en un servidor se lleva a cabo a través de la implementación de formularios HTML desde el lado del cliente, como se analizó en el Capítulo 4 de este módulo.

Los dos puntos clave para el modelado de formularios son: el **atributo METHOD**, que indica el **método de envío seleccionado del protocolo de comunicación HTTP**, y el **atributo ACTION**, donde se indica la **redirección al fichero de destino php**.



```
<FORM ACTION="destino.php" METHOD="GET">
    //acciones
</FORM>
```

Código 8. Diseño formulario con método GET

A continuación, se realiza el modelado habitual de un formulario con HTML. Es imprescindible que **para la creación de cada uno de los campos que conforman los formularios se les asocie un nombre** (atributo **NAME**) puesto que esta será forma de identificarlos en el envío de datos y su posterior recuperación.

```
<FORM ACTION="destino.php" METHOD="GET">
Nombre: <input type="text" name="nombre"></br>
Asunto: <input type="text" name="asunto"></br>
Mensaje:<input type="text" name="mensaje"></br>
<input type="submit" value="OK" />
</FORM>
```

Código 9. Ejemplo diseño formulario con método GET

El formulario quedaría así:

Nombre:

Asunto:

Mensaje:

Fig. 7. Formulario salida código 9

Por lo tanto, en este caso se almacenaría en el servidor los datos recogidos en el formulario definido.

/ 6. Controles con verificación de información remota en el cliente

La verificación de la información incluida por el cliente se realiza de dos maneras posibles.

Desde el lado del cliente es posible añadir ciertas comprobaciones utilizando la tecnología JavaScript, aunque estas no son consideradas un método fiable de comprobación. No obstante, sí resultan útiles para realizar un filtrado previo de peticiones, con el objetivo de no saturar el servidor con datos incorrectos.

Algunas de estas comprobaciones en el lado del cliente pueden ser:

- El **número de caracteres** es incorrecto/correcto.
- **No se han introducido todos los valores especificados para un determinado campo**, por ejemplo, una contraseña que deba estar formada por un número concreto de caracteres o que incluya cierto tipo de símbolos.



Pero además de estas validaciones en el cliente, **será imprescindible implementar el código necesario para en el servidor se realicen las validaciones de datos oportunas para garantizar la integridad de los datos que puedan provenir de fuentes inseguras.**

En el lenguaje de programación PHP se incorporan múltiples funciones que permiten realizar este filtrado.

Función	Definición
<code>filter_has_var()</code>	Esta función permite comprobar si existe una variable que sea de un tipo concreto.
<code>filter_var()</code>	Esta función permite filtrar una variable indicando el tipo de filtro exacto.
<code>filter_var_array()</code>	Esta función permite filtrar un array de variables indicando el tipo de filtro con el que va a ser filtrada cada variable.
<code>filter_input()</code>	Esta función valida el valor de un campo en función del filtro especificado.
<code>filter_input_array()</code>	Esta función valida el valor de los campos de entrada recogidos en el array y permite utilizar un filtro para cada uno de ellos.

Tabla 3. Funciones filtrado PHP

6.1. Filtros de validación y saneamiento

Filtros de validación

En las funciones señaladas en la tabla anterior se permite utilizar de forma opcional, **filtros que evalúan las variables de entrada**. Estos filtros quedan caracterizados por un nombre identificativo en función del tipo de filtrado que aplican. Algunos de los más comunes se muestran en la siguiente tabla:

Filtro	Definición
<code>FILTER_VALIDATE_EMAIL</code>	Valida una dirección de correo electrónico.
<code>FILTER_VALIDATE_BOOLEAN</code>	Valida si el valor es de tipo boolean. Si es true devuelve 1 y si es false devuelve 0.
<code>FILTER_VALIDATE_FLOAT</code>	Valida si el valor es de tipo float.
<code>FILTER_VALIDATE_INT</code>	Valida si el valor es de tipo int.
<code>FILTER_VALIDATE_IP</code>	Valida si el valor es una dirección IP.
<code>FILTER_VALIDATE_URL</code>	Valida si el valor es una URL de acuerdo con el estándar RFC 2396.

Tabla 4. Filtros de validación



Filtros de saneamiento

Además de estos filtros, existen otras opciones de filtrado conocidas como “de saneamiento” que permiten **limpiar ciertos valores de entrada**, como, por ejemplo:

Filtro	Definición
FILTER_SANITIZE_EMAIL	Elimina todos los carácter menos letras, dígitos y <code>!#\$%&'*+,-=?^_`{ }~@.[]</code> .
FILTER_SANITIZE_ENCODED	Elimina o codifica caracteres especiales.
FILTER_SANITIZE_NUMBER_FLOAT	Elimina todos los caracteres excepto los dígitos y los caracteres <code>+</code> y <code>-</code>
FILTER_SANITIZE_STRING	Elimina etiquetas y puede eliminar y codificar caracteres especiales.

Tabla 5. Filtros de saneamiento

6.2. Filter_input y filter_has_var

Filter_input

Finalmente, analizaremos en detalle algunas de las funciones de filtrado claves para la verificación de los datos enviados.

La función **filter_input()** es una de las más importantes para el proceso de **verificación de los datos que puedan provenir de fuentes inseguras**. La sintaxis completa de esta función es la siguiente:

```
filter_input(type, variable, filter, options)
```

Código 10. Uso filter_input

El parámetro **type** es de carácter obligatorio y se utiliza para indicar el tipo de valor de entrada que se debe comprobar/verificar.

Algunos de estos valores son:

- **INPUT_GET**. Datos de entrada recibidos a través del método GET, `$_GET[]`
- **INPUT_POST**. Datos de entrada recibidos a través del método POST, `$_POST[]`
- **INPUT_COOKIE**. Datos de entrada recibidos a través de COOKIE HTTP
- **INPUT_SERVER**. Datos de entrada recibidos a través del `$_SERVER[]`. Estos valores están asociados a la información del entorno del servidor (cabeceras, rutas, ...).
- **INPUT_ENV**. Datos de entrada recibidos a través método del entorno `$_ENV[]`.



Audio 1. “Parámetros obligatorios y opciones en filter_input”
<https://bit.ly/3eppCSC>



Filter_has_var

La función **filter_has_var()** se utiliza **para comprobar si la variable que recibe por parámetro es de un tipo concreto (type)**.

Esta función retorna un valor de tipo **booleano**, true si la variable es del tipo indicado o false en caso contrario.

```
filter_has_var(type, variable)
```

Código 11. Uso filter_has_var

El resto de las funciones descritas en la Tabla 3, así como alguna más, se pueden consultar desde la página web oficial del Manual de PHP ([enlace](#)).

6.3. Diseño de un caso de verificación con formulario

Para verificar los datos recibidos del servidor, utilizando las funciones de filtrado recogidas en la tabla 4 para PHP, será necesario incluir en el **fichero de implementación del formulario el código en lenguaje PHP que permite el filtrado y validación de los datos**.

Cómo se puede ver en el siguiente código de ejemplo, la evaluación de condiciones desarrollada como PHP queda embebida en el código HTML. En este caso, solo se ha incluido la validación del campo *nombre*, pero podría situarse también para la evaluación del resto de valores.

```
<html>
<head> ... </head>
<body>
<FORM ACTION="destino.php" METHOD="GET">
Nombre: <input type="text" name="nombre"></br>
<?php
if (!filter_has_var(INPUT_GET, "nombre")) {
echo "No se ha introducido ningún valor";
}else{
if (!filter_input(INPUT_GET, "nombre", NOMBRE_VALIDACION)) {
echo "El nombre de usuario introducido es incorrecto";
}else{
echo "El nombre es correcto";
}
}
?>
Mensaje:<input type="text" name="mensaje"></br>
<input type="submit" value="OK" />
</FORM>
</body>
</html>
```

Código 12. Ejemplo diseño formulario con método GET y validaciones

La función **filter_has_var()** recibe por parámetro el contenido del elemento **nombre** y comprueba si está vacío, en caso de ser así devuelve un mensaje. Ahora bien, cuando se ha introducido un valor se verifica su contenido utilizando la función **filter_input()**. En este caso se ha utilizado una variable de filtro llamada **nombre_validación**.



Video 2. "Formulario con verificación de los campos"

<https://bit.ly/38aDcbI>





/ 7. Caso práctico 1: “Uso del elemento Location”

Planteamiento: Se pide diseñar el código necesario para implementar el siguiente comportamiento interactivo:

1. Si se pulsa el botón *continuar* de la primera pantalla aparece un cuadro de diálogo.
2. Al pulsar el botón *sí* nos llevará a la *pantalla A*. Se envía por parámetro una variable de nombre *var* y valor *A*.
3. Al pulsar el botón *no* nos llevará a la *pantalla B*. Se envía por parámetro una variable de nombre *var* y valor *B*.
4. Utiliza el elemento *Location*.

Nudo: Para crear el cuadro de diálogo que permite al usuario determinar el camino a seguir se va a utilizar la función *confirm*, ésta recibe por parámetro el texto que va a ser mostrado al usuario. Habitualmente las preguntas modeladas serán respondidas con *sí* o *no*, que serán evaluadas como *true* o *false* en la estructura de control *if* que se muestra en el código de solución siguiente:

Desenlace: En el siguiente fichero de origen (Pantalla origen) si el usuario pulsa el botón “*cambiar de pantalla*” será redireccionado a una nueva página (Pantalla A o B).

```
<html>
<head>
<title> Pantalla 1 - ORIGEN </title>
<script type="text/javascript">
function siguientePantalla(){
if(confirm('Si pulsas SI vamos a la pantalla A. Si pulsas NO vamos a la
pantalla B. Elige bien')){
location.href='pantallaA.php?var=A';
}
else{
location.href='pantallaB.php?var=B';
}
}
</script>
</head>
<body>
<p onclick="siguientePantalla()"> CAMBIAR DE PANTALLA </p>
</body>
</html>
```

Código 13. Código Caso práctico 1

/ 8. Caso práctico 2: “Crea un formulario verificador”

Planteamiento: Se pide diseñar un nuevo formulario con 2 campos de texto: *nombre de usuario* y *contraseña* para poder acceder al resto de servicios de la página web. Se va a implementar una nueva página web con HTML y PHP para las verificaciones oportunas:

- Se va comprobar que se ha introducido un valor en el campo del *login*.
- Utiliza un método *POST*.



Nudo: La secuencia de pasos en cuanto a la implementación del nuevo fichero de formulario verificador será:

1. Se crea los campos de texto indicados.
2. Se añade cada uno de estos campos la validación con PHP.

Desenlace: El código resultante se muestra a continuación:

```
<html>
<head>
<title> FORMULARIO - VERIFICADOR </title>
</head>
<body>
<FORM ACTION="validar.php" METHOD="POST">
Login: <input type="text" name="login"></br>
<?php
if (!filter_has_var(INPUT_POST, "login")) {
echo "No se ha introducido ningún valor";
}else{
if (!filter_input(INPUT_POST, "login", NOMBRE_VALIDACION)) {
echo "El login de usuario introducido es incorrecto";
}else{
echo "El login es correcto";
}
}
?>

Pass:<input type="text" name="pass"></br>
<input type="submit" value="Verificar" />
</FORM>
</body>
</html>
```

Código 14. Código Caso práctico 2

/ 9. Modificación de la estructura de una página

En la actualidad, una de las maneras más utilizadas para modificar la estructura de una página web interactiva es a través de **la estructura de árbol DOM (Document Object Model)**, que permite actuar de forma dinámica sobre los elementos de esta.

Como seguro que ya sabes, este modelo diferencia entre varios tipos de elementos, denominados **nodos bajo el modelo DOM**.

- **Document.** Se trata del nodo raíz del que derivan el resto.
- **Element.** Representa las etiquetas utilizadas en HTML, puede contener atributos, los cuales coinciden con los utilizados en cada etiqueta.
- **Attr.** Representa los atributos de las etiquetas HTML.
- **Text.** Representa el texto que se encuentra entre los delimitados de una etiqueta HTML.
- **Comment.** Se utilizan para representar los comentarios en una página HTML.



Este modelo utiliza un sistema de nodos que quedan identificados de diferentes formas, por ejemplo, a través del nombre de elemento, de un identificador o de un atributo, entre otros. El acceso a cada uno de estos elementos es posible utilizando las siguientes funciones:

- **getElementsByTagName(Etiqueta):** Obtiene todos los elementos de un documento HTML cuya etiqueta sea indicada por parámetro. Esta función se aplica al elemento “document”, es decir, a todo el documento HTML.

```
var elemP = document.getElementsByTagName("p");
```

Código 15. Función acceso por etiqueta a elemento

- **getElementsByName(Etiqueta):** Obtiene todos los elementos de un documento HTML cuyo nombre (atributo *name*) coincida con el que se indica por parámetro. Esta función se aplica al elemento “document”, es decir, a todo el documento HTML.

```
var elemName = document.  
getElementsByName("prueba");
```

Código 16. Función acceso por atributo a elemento

- **getElementsById(Etiqueta):** Obtiene todos los elementos de un documento HTML cuyo identificador coincida con el que se pasa por parámetro. Esta función se aplica al elemento “document”, es decir, a todo el documento HTML.

```
var elemName = document.getElementById("id");
```

Código 17. Función acceso por id a elemento

Además del acceso a los nodos ya creados y la modificación de su valor, es posible crear nuevo contenido utilizando las funciones dedicadas a tal fin: *document.createElement(Etiqueta)*, *document.createTextNode("cadena de texto")* y *element.appendChild(elemento_hijo)*.

/ 10. Resumen y resolución del caso práctico de la unidad

A lo largo de la unidad, hemos podido comprobar que una **página web interactiva** es aquel sitio en el cual existe una interacción activa entre cliente y servicio, donde el usuario realizará diferentes tipos de peticiones y la página web le devolverá la respuesta oportuna.

Hemos estudiado también, que la **redirección** de páginas a través del lenguaje de programación PHP, resulta una herramienta clave en el desarrollo de páginas web interactivas. Para implementar este funcionamiento será necesario desarrollar tantos ficheros PHP como páginas o secciones se quieran mostrar a un usuario ante las tomas de decisión y la interacción que este realiza sobre el sitio web.

Para esta redirección es imprescindible el uso de un nuevo tipo de objeto, conocido como **Location (window.location)** que será el encargado de implementar la redirección a otra página web. Algunas de las propiedades de este objeto son: Además de indicar el fichero destino (nombreDestino.php) a través de la propiedad **href** del objeto **Location**, será posible el envío de variables de forma opcional.

También hemos aprendido que la **verificación de la información** incluida por el cliente se realiza de dos maneras posibles. En primer lugar, desde el lado del cliente es posible añadir ciertas comprobaciones utilizando la tecnología JavaScript, aunque estas no son consideradas un método fiable de comprobación, no obstante, sí resultan útiles para realizar un filtrado previo de peticiones, con el objetivo de no saturar el servidor con datos incorrectos. Pero además de estas validaciones en el cliente, será imprescindible implementar el **código necesario para que en el servidor** se realicen las validaciones de datos oportunas para garantizar la integridad de los datos que puedan provenir de fuentes inseguras.

Resolución del caso práctico inicial

Además de la recepción de datos en el servidor, es imprescindible la implantación de la funcionalidad necesaria para que estos datos sean validados en el destino, ya que las fuentes pueden ser inseguras. De esta forma, cuando se validen los datos del proceso, el usuario podrá enviar datos al servidor donde quedarán almacenados, para que posteriormente puedan ser utilizados en múltiples acciones.

Por ejemplo, será posible recuperar las sesiones almacenadas para cada uno de los usuarios autenticados en un sitio web, si los datos son correctos, puesto que previamente éstos se habrán enviado al servidor donde han quedado almacenados.



Fig. 8. Comprobaciones de datos

/ 11. Bibliografía

Ganzabal, X. (2019). *Desarrollo web en entorno servidor* (1.a.ed.). Madrid, España: Síntesis

Webgrafía

PHP Manual. Consultado en: <https://www.php.net/>

Filtros para PHP. Consultado en:

<https://www.php.net/manual/es/filter.filters.validate.php>